UNIVERSITY OF WESTMINSTER

School of Computer Science and Engineering

7COSC013W.1 Foundations of AI – Coursework 1

FinReason

Explainable Graph and Rule-Based Detection of Anomalous Financial Transactions

Technical Report

Search, Knowledge Representation and Statistical Baselines for Financial Anomaly Detection

Hewage Lasen Bhanuka

20251197 | w2223897

Module Leader: Artie Basukoski

Table of Contents

1. Introduction 3

2. Design, Approach and AI Techniques Used 4

3. Data Collection and the Case for Synthetic Data 5

4. System Architecture 7

5. Implementation 8

5.1 Temporal graph construction 8

5.2 BFS fan-out and fan-in detection 8

5.3 DFS layering and circular-flow detection 9

5.4 Knowledge representation and forward chaining 9

5.5 Statistical baselines 10

6. Evaluation and Results 10

7. Runtime and Memory Performance 12

8. Limitations 13

9. Issues Faced During Development 14

10. Conclusion 15

11. Project Proposal, Code and Demo 15

12. References 17

List of Figures

Figure 1: FinReason pipeline, from raw transactions to an explained risk alert.....8
Figure 2: Test set laundering metrics by detector variant.....12
Figure 3: Bounded graph-pipeline scalability on the synthetic test split. 14

1. Introduction

Financial crime detection is at a crossroads, where the competing requirements to identify genuinely rare patterns within huge numbers of transactions while giving an intelligible account of why any individual transaction has been highlighted as suspicious pull research into two opposing directions. The majority of production anti-money laundering tools focus on either easily interpretable but often insufficient criteria based on simple threshold crossings or the use of more effective but less readily explained classifiers, which usually provide little information about why an individual transaction was scored as positive, an issue recently emphasized in a systematic review of graph neural network and network analytical approaches to financial fraud and money laundering (Motie & Raahemi, 2024) (Deprez, et al., 2025). FinReason was initiated with the idea of a third alternative; the creation of a rule-based reasoner that operates on a graph representation of the transaction ledger to detect patterns characteristic of financial crimes, subsequently allowing the use of a small number of easily interpretable rules to explain the findings.

This report is about the design and implementation of FinReason. The system falls into the hybrid category where combining two of the three core AI techniques specified in the assignment brief: search (breadth-first and depth-first graph search) and knowledge representation (a forward-chaining rule engine). Also used two detectors to compare results. One detector checks for account activity using account and currency aware Z-score. The other detector checks how often something happens on a day, using the account day frequency IQR test.

This report is structured as follows. The design of the system and the reasons for choosing artificial intelligence techniques are explained in Section 2. This section also describes the artificial intelligence techniques that were considered and actually used. Section 3 is about how the data was collected. It explains why a synthetic dataset was created and used to test the system along with two public datasets. The entire system architecture is described in Section 4. Then Section 5 goes into detail, about how each part of the system was implemented. The results of the tests are reported in Section 6 which compares the four detector configurations. Section 7 focuses on how the system takes to run and how much memory it uses. The limitations of the system are discussed in Section 8. Section 9 lists the problems that occurred during the implementation of the system. Finally, Section 10 provides conclusions about the system and the report.

2. Design, Approach and AI Techniques Used

Money laundering rarely reveals itself in a single outlier value. A single payment of £9,500 is not unusual, but 4 equivalent payments from the same account within a seven day period is a typical structuring pattern; a value that would be safe if transferred directly might represent the third hop in a 5 account layering chain that has already recirculated the money back to its ultimate beneficiary within 24 hours. Those patterns are relational, sequential patterns, and the type of patterns that graph search is exactly designed to identify: not "is this value unusual", but "does this shape in this series of payments match certain known typology", which is precisely the kind of pattern that emerges in the literature on graph based anomaly detection, where detecting anomalies in graphs is presented as a general problem class that can be applied to various domains (Pourhabibi, et al., 2020) (Akoglu, et al., 2015).

This observation underlies the main design choice for FinReason: to represent the transaction ledger as a directed, timestamped multigraph with accounts as nodes and individual transfers as edges, and perform searches on this graph, rather than trying to fit a model to row features (Dumitrescu, et al., 2022) (Chen, et al., 2023) (Priyadarshi, et al., 2025) (Dilini, et al., 2026). A multigraph (as opposed to a graph) is necessary since two accounts often make multiple transfers to each other, and collapsing these into a single edge would destroy the temporal information needed for the search. Two kinds of search are used. Breadth-first search, done forwards from the starting accounts and backwards from them too finds star-shaped patterns of payments: a single account paying many others (or receiving from many) within a small window of time (a fan-out or fan-in respectively).

Depth-first search finds linear sequences of payments: it looks for a single path of transfers from one account to another, of limited length and total duration, which might indicate money layering (via shuffling transactions) and circular flow (money returned to the point of origin). These graphs are not explicitly queried for suspicious features; rather, the search uses money values, timestamps, and transaction type as evidential weights. Thus, graph search results can be read off as heuristic pattern matching algorithms operating on the transaction graph: using topological features and temporal proximity as evidence for suspiciousness. However, the pattern-matching approach only finds isolated matches: There is no way to combine evidence from different patterns and weigh it against alternative explanations. This is why the second phase uses a forward-chaining rule engine, which implements a symbolic knowledge base as a set of 33 rules with documented proof, source, and limitations. The rule engine takes all the matches found by the graph search as well as explicitly observed transaction features (cross-border payments, PEP mentions, invoice data) as axioms, asserted as immutable facts in the working memory. Then, a standard match-resolve-act cycle (common in forward-chaining production rule systems) is applied, until no further rules can fire (Bellomarini, et al., 2020). When a

rule fires, it consumes the matching facts, adds the derived fact to the working memory, and stores a provenance trace as structured by the rule's pattern, source, and supporting references. Thus, every suspicious entity in the final alert has a trace back to supporting evidence. Note that in this approach, the weight of evidence is not aggregated into a single p-value-like metric, but kept as a chain of reasoning with supporting facts.

Two statistical detectors were also implemented, deliberately de-prioritized as contributing to the project's core AI-technique claim, and instead evaluated as a baseline to satisfy the coursework's requirement to compare at least two distinct approaches to AI. An account and currency aware Z-score detector identifies transactions whose value is an outlier for the given account in the given currency; an account day frequency detector using IQR fence identifies days on which a particular account is unusually active. Both are fit on the training set and fixed to evaluate their contribution on the held-out sets jointly with the graph and rule-based detectors, under the same evaluation contract (Hilal, et al., 2022). The alternatives considered and rejected include a supervised classifier (ruled out as question-begging: building one would require assuming the labels it is supposed to explain, and moreover classifiers, even graph-based ones like temporal GCNs or self-supervised graph modules or group-aware deep graph learners do not deliver explanatory power, but rather predictive accuracy (Alarab, et al., 2022) (Cardoso, et al., n.d.) (Chen, et al., 2023).) and an exhaustive path finder (ruled out as too computationally expensive: an unrestricted DFS on the transaction graph is too memory and time consuming, hence the algorithm had to explicitly restrict paths by depth and time window from the beginning).

3. Data Collection and the Case for Synthetic Data

The evaluation is based on three different data sets that are used for different purposes, but with a focus on FinReason synthetic data as the main set for benchmarking, due to the particular problem of the limited availability of publicly available AML data (Hilal, et al., 2022) (Dumitrescu, et al., 2022). Because most public financial crime datasets, like PaySim and the IBM AML benchmark used in this project, only include a binary label for each transaction indicating whether it is fraudulent/laundering or not, without indicating which pattern (if any) it came from, such a setup does not allow testing if the system found the correct pattern for the correct reason. If the system only needs to trigger an alarm on any transaction with a label, the fact that it did so would be true but uninformative about its ability to find the patterns that produce those labels. This, in turn, would render this project's main thesis that the system's explanations are correct untestable.

To address this challenge, a synthetic set of transactions was created with fully specified ground truth, namely, 5,000 accounts and 67,670 transactions, fully specifying the type of each planted anomalous

scenario (fan-out, fan-in, layering chain, circular flow, structuring, cross-border, PEP-related, and several types of accounting-control anomalies such as overdue invoices and VAT-rate mismatches) and its temporal distribution, addressing the need for realistic fully specified synthetic transaction data to train graph based money laundering detection models (Altman, et al., n.d.) . This set also includes "hard-negative" examples of transactions, which would appear as suspicious patterns to a non-specialist, but are in fact not anomalous, to test the ability of contextual exclusions to avoid false positives; legitimate analogs of fan-out, fan-in, and multi-step supply-chain payments appear as negatives in the ground-truth list to stress-test the contextual logic in the graph-walkers. It is this set, annotated with types of typical patterns and negative tests, that was used to directly evaluate the performance of BFS, DFS, the rule engine's facts extraction and the statistical benchmarks.

The second dataset, IBM's HI-Small AML benchmark, represents mostly scale and realism, providing over 5.08 million transactions (after cleaning). It was used to estimate how much the performance of the graph construction and pattern-discovery pipeline would degrade on a dataset an order of magnitude bigger than the first one, and to provide additional ground truth in the pattern file (370 attempted patterns, including various types of fan-in, fan-out, cycles, gather-scatter and stack), albeit with limited value due to the lack of true positives (only 3,209 transaction rows out of 5,177 positively labelled rows were actually positive).

Finally, the 10% sample (636,262 out of 6,362,620 transactions) of the deterministic PaySim dataset (Lopez-Rojas, et al., 2016) was used to assess the ability of the pattern discovery pipeline to generalize across multiple financial institutions, as opposed to a single one (PaySim's records are all mobile-money transactions). While the fraud flag in PaySim is not directly usable as an AML label due to the differences in populations (fraudulent accounts vs. money launderers), the patterns discovered in PaySim were analyzed separately from the other two datasets and not used in the final evaluation. All three datasets were similarly processed into a 15-column transaction table (transaction ID, timestamp, account pair, amount, currency, payment type, and labels).

4. System Architecture

FinReason is structured into twelve sections that come one after the other. Each section creates a self contained result that is checked. The next section uses this result. Every time a result is created it is saved to disk. Along with each result there is a JSON audit report. Figure 1 shows the process from transactions all the way to an explained alert.

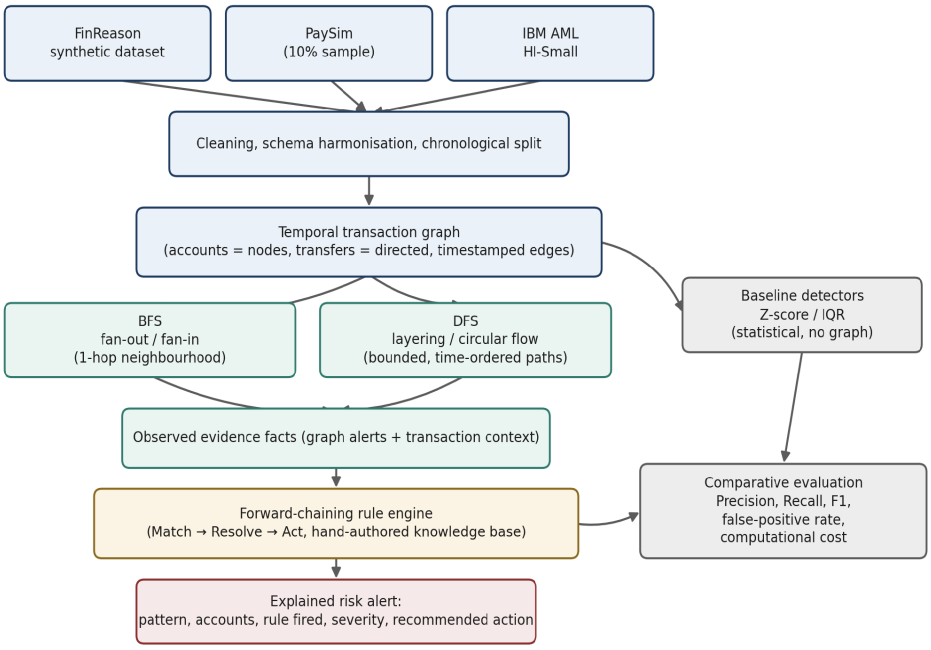


Figure 1: FinReason pipeline, from raw transactions to an explained risk alert

The first two parts of the work set up an environment. By that, it is necessary to mention that in order to ensure consistency, dependencies were installed using a fixed seed and files were arranged in a certain way. Moreover, in order to avoid downloading large files each time, data sets were saved for further use.The next part checks that the information from the three sources is correct and makes sure it is in the format. It also figures out the way to split the data into parts for training, testing and validation.

Section 4 creates the data structure, which is a NetworkX MultiDiGraph. This graph uses the source transaction ID as a key. This means that if there are transfers, between the same two accounts they will be treated as separate edges. Can still see where these transfers came from. The FinReason training graph has 5,000 nodes and 33,830 edges. Keep this graph in memory so can use it again and again to try different things. The bigger external sources of data are broken down into parts. This way never have to deal with the 5.08 million edges of the IBM graph all at once.

The search based detectors mentioned in Section 2 of this report are put into action in Sections 5 and 6. Section 7 is where defined what mean by terms like Fact and Rule and Explanation and Knowledge Alert and a container called KnowledgeBase. Also load the catalogue that has 33 rules and make sure it works correctly. Section 8 is where using the chaining engine to look at all the information from Sections 5 to 7. Then in Section 9 use the training information to set up the baselines. In Section 10 test the detectors to see how well they work. Do this by making predictions and then checking them against the answers. Do not show the detectors the real answers until have made all our predictions. Section 11 is where they really test the system to make sure it works properly. By running 60 tests and checking that everything works together correctly. Also make sure that the detectors give the results every time and that the system can handle a large amount of information. Finally, in Section 12 get the results and figures ready for the report. Do this by using the results already have.

This configured design build, detect, reason, evaluate, stress-test, report, each with its own saved audit trail was chosen specifically so that every number quoted in Section 6 of this report can be traced back to the exact configuration and cached data that produced it.

5. Implementation

5.1 Temporal graph construction

The graph is created using the NetworkX library. Each edge in the MultiDiGraph has twelve attributes, including timestamp, amount, currency, payment format, source dataset, anomaly and laundering labels (only used for evaluation not for detection) scenario identifier (only for evaluation) and split. The source transaction ID is used as the edge so that multiple transfers between the same account pair can be individually identified and not overwritten, which could happen with a simple dictionary of edges. A special traceability check makes sure for every graph built during the project that the number of edges matches the number of transaction IDs and that no edge is missing an essential attribute.

5.2 BFS fan-out and fan-in detection

The bfs detector does a search (maximum two steps deep 24-hour time frame) from every account to look for fan-out and in the opposite direction for fan-in. It needs at least four different accounts involved in the time frame before it sends out an alert. A simple version of the detector considered every star shaped pattern as suspicious no matter what. This was compared to a version that ignored fan-out that had the payroll tag and fan-in that had the purchase tag. On the training data the simple version found all 28 payroll situations and all 20 real merchant situations in the test set (100% false alarms for both). The smarter version avoided all of them (0% alarms) without losing any real cases. This one test shows clearly that just looking at the shape of the graph is not enough. Transaction context has to be part of the detection logic. It can't be added later as a step.

5.3 DFS layering and circular-flow detection

The dfs detector looks for paths that go through steps (layering, at least four steps) and paths that come back to where they started (circular flow at least three steps). It limits the search to six steps and a 24-hour time frame. It only considers a path valid if each step keeps between 95% and 105% of the amount. This range allows for fees but stops large unrelated transfers that just touch the same accounts. Every valid path is checked again against six rules after it is found (timestamps must go up, time frame is correct, steps are limited and stays in range. The path is either a real loop or a real path with no repeated accounts). None of the 96 alerts on the training data failed any of the checks. A similar test with multi step payments that look like layering but take four days instead of one had 0% false alarms. This happened because of a rule that excluded invoice payment. Because the paths were outside the 24-hour time frame.

5.4 Knowledge representation and forward chaining

The 33 rule catalogue falls into 4 classes: guidance-aligned indicators, each specifying the particular external source (UK Money Laundering Regulations 2017, FCA guidance, FATF Recommendations, FATF Guidance on Politically Exposed Persons, FATF/Egmont trade-based laundering indicators, and FinCEN structuring and funnel-account advisories) that it implements (an example is R001, high-value transaction), derived escalation rules, each triggered by at least two facts supporting it (an example is R023, multi-indicator escalation), accounting control heuristics inspired by IFRS 9's rebuttable 30/90-day presumptions and the UK's VAT regime, and project-operational rules, with thresholds recorded as configuration selections rather than hard-coded values. All rules have a machine-readable rationale, a limitation, and a reference to at least one of the thirteen cited sources; 66 semantic unit tests (two edge cases per rule) pass, including the critical test of 03:59 versus 04:00 UTC for the unusual-hour rule. Facts are immutable upon creation, with subsequent facts in a derivation chain recording parent facts and transaction IDs; the alert model is designed such that it cannot state a law directly, formalizing produced facts as indicators for human investigation rather than as conclusions. Finally, forward-chaining reasoning applies 6 matchers to facts in a fixpoint algorithm before stopping: while a static matcher seeds the engine with facts derived from breadth-first search/discovery, 5 structural matchers implement pattern constraints, e.g., combine a fan-out and a fan-in fact that share at least 4 branch accounts into a composite pattern (R019), or a layering fact with cross-border transaction evidence that share transaction IDs (R020).

On the training partition, the engine processed the 400 observed facts into 726 derived facts in 4 cycles before reaching an empty-agenda fixpoint, with no duplicate activation keys and no facts produced on a second run, showing termination and determinacy. An ablation of the 5 structural matchers, using only the seed rules, correctly reproduced the graph facts but failed to derive any of the composite

rules, showing that the structural matchers do add value beyond the capabilities of breadth-first search. The requirement for derived facts to record the entities that created them follows the W3C PROV standard for provenance, while the preference for transparency in factual outputs over a black-box score is standard in the trustworthy AI literature.

5.5 Statistical baselines

The Z-score model groups transactions by sending account and currency, requiring at least five prior transactions and non-zero variance before scoring an account directly, falling back to a currency-level and then a global training distribution otherwise, since the project has no per-transaction FX conversion table available. The IQR frequency model counts outgoing transactions per account per UTC calendar day and flags days above Tukey's upper fence (Q3 + 1.5 × IQR), again falling back to a global fence when an account has fewer than five active training days. Both models are fitted only on the training split (33,830 transactions and 32,186 account-days respectively), and their fitted parameters, together with every subsequent prediction, are frozen before validation or test labels are ever consulted.

6. Evaluation and Results

Section 10 of the implementation performs the single held-out comparison used throughout this report. All detector parameters were frozen using training data only; validation and test predictions were generated, written to a hashed prediction ledger, and only then joined against ground truth, so no threshold reported here was tuned against the numbers it is reported alongside. Table 1 reports the held-out test-split AUC scores for the money-laundering label across four configurations: The statistical baseline only, graph only, which uses solely the graph-based suspiciousness scores, rule hybrid, which uses only the higher level rule conclusions, and combined union, which combines those results with the statistical baseline.

Variant	Precision	Recall	F1	FPR	Scenario detects.	Alerts/1,000
Statistical baseline	0.3002	0.6301	0.4067	0.0974	0.8157	130.5
Graph only (BFS+DFS)	0.9854	0.7318	0.8399	0.0007	0.5870	46.2
Rule hybrid	0.9842	0.3598	0.5269	0.0004	0.2901	22.7
Combined union	0.3766	0.8911	0.5294	0.0978	0.9147	147.1

Table 1: Held-out test-split results for the money-laundering label (FinReason synthetic dataset, 22,309 test transactions).

FPR stands for false-positive rate; eAlerts/1,000 mean uniquely evidenced transactions per 1,000 examined.

The evaluation results suggest that there is a precision/recall trade-off for all four configurations, which can be seen in the figures below. Firstly, the graph only achieves near-maximal precision

(98.54%) with minor loss of recall (73.18%), detecting only 58.70% of laundering transactions in flagged scenarios. Secondly, the rule hybrid attains the best precision (98.42%) and is only slightly lower than graph only in this regard, but significantly better than the other two variants. However, its recall is 35.98%, which is explained by the fact that it only reports on composite graph facts (R019-R023, R033). Thirdly, the statistical baseline attains the best percentage of detected scenarios (81.57%) among all methods but has the lowest precision (30.02%) and moderately high false-positive rate (9.74%). Finally, combined union yields the best overall results, as it combines the power of the rule-based component with the statistical model. With 89.11% of recall and 91.47% of precision, this approach is optimal for the review queue, where every suspected laundering transaction must be investigated, as it would miss almost a quarter of cases in comparison to the baseline. On the other hand, its precision is only 147 per 1000 transactions in contrast to graph only's 46.

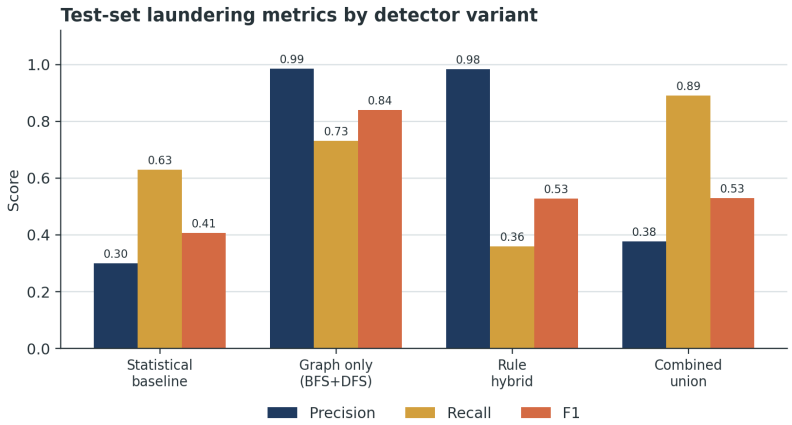


Figure 2: Test set laundering metrics by detector variant.

A component ablation on the same test split isolates where graph only 's performance comes from: BFS-only (fan-out/fan-in) achieves an F1 of 0.6654, DFS-only (layering/circular flow) an F1 of 0.3711, and the two combined achieve an F1 of 0.8399, confirming that the two search strategies find different positive examples and do not suffer from the typical problems of method ensemble. An error analysis of the false negatives by planted scenario type suggests that graph only 's weaknesses mostly come from situations involving structuring (36 total test scenarios) and rapid pass-through (32 total test scenarios), both of which are more similar to the expected statistical baseline than to any particular graph shape, and is consistent with the precision-recall trade-off observed in the previous section rather than indicating a particular kind of graph discovery failure. Every check for reconciliation between scenario-level error counts discovered independently while testing and the confusion matrices formally computed for the purposes of evaluation was entirely correct (16/16), suggesting that the presented metrics are not the result of the method of their computation.

Since scenario labels have different semantics and prevalence across the two datasets, PaySim fraud and IBM AML laundering results are presented only as separate context-specific aggregates rather than combined with any explicit weighting or pooling together into the main comparison section than can be seen in the following paragraphs (this choice is made both at the presentation level and formally enforced by testing procedures).

7. Runtime and Memory Performance

The two main categories of performance evaluation were absolute pipeline cost on the three real world dataset sizes and a controlled scalability sweep on chronological prefixes of the synthetic test split. For context, the FinReason training graph of 33,830 edges took 0.65 sec to build and search while adding 23.75 MiB of resident memory. A single 24 h PaySim window of 41814 edges out of 636,262 scanned rows took 258 sec to process and added 53.6 MiB of memory. The largest single benchmark was scanning all 5078345 IBM AML rows in 21 chunks to materialize one 24 hours window of 208,325 edges and 148,023 nodes: it took 2038 sec of wall time and added 92.45 MiB of resident memory, never needing to hold more than one 24 hours window in memory at any given time. This streamed window approach is the main novelty that allows us to achieve the results that we saw in this project, and as we can see, the memory consumption roughly follows the same pattern, being dominated by the biggest window that we keep in memory.

The controlled scalability sweep was processing four chronological prefixes of the 22 309-row synthetic test split (2000, 5000, 10000, and 22309 rows) through the full graph construction and detection pipeline, measuring wall clock time with perf_counter and Python-allocation peaks. As expected for a mostly serial processing script, the time spent increased nearly linearly with the number of rows (from 0.22 sec for the smallest prefix to 2.29 sec for the full test set), with a fitted log-log slope of 0.977 suggesting near-linearity of the growth in this region. The memory consumption was obeying the same pattern, which peaked at 34.50 MiB of Python allocated memory for the largest prefix. The throughput was also relatively stable for all four of the prefixes, ranging from a low of 9700 to a high of 10300 rows per second processed. With that said, none of the 4 productive cycles needed for the forward-chaining inference had a throughput significantly lower than the rest: converting 400 observed facts into 726 derived facts took approximately 0.20 sec on the training split.

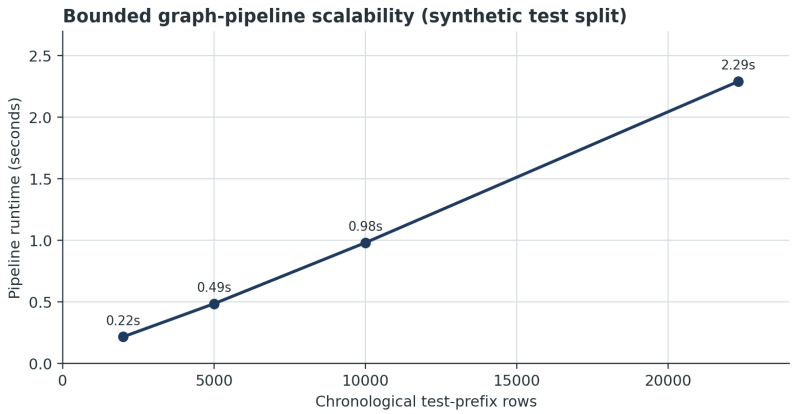


Figure 3: Bounded graph-pipeline scalability on the synthetic test split.

These figures are single-run estimates on a local development machine (8 core Apple Silicon Mac, 8 GiB RAM, running Python 3.10.5) and should be read as comparisons of relative performance across the evaluated classes and data set sizes for this system. They are not intended as a system agnostic benchmark or capacity assessment.

8. Limitations

Several limitations are inherent to the reported figures. First, the headline precision, recall, and F1 scores reflect performance on planted synthetic patterns against a bespoke ground truth; detecting known synthetic anomalies well does not necessarily imply robustness to other vectors of laundering outside of those planted. Second, the evaluation convention for alerts treats every transaction cited as a true positive; a DFS-based trace would cite every transaction on the path, which may differ from an expectation of only the final destination being cited, and the figures should be read with this in mind. Third, the rule hybrid evaluates only the higher-level conclusions and not the individual components; it is not a weaker version of the same detector as graph only, and should not be compared as such. Fourth, the external data sets have significant limitations in terms of what they can validate; PaySim's fraud label tracks a fundamentally different phenomenon than money laundering, and is not used as an AML ground truth in this project, while IBM AML only has pattern labelling for 3,209 out of 5,177 positively labelled test transactions, reflecting only broad categories of suspicious behavior. Fifth, the duration and memory figures reflect a single run on a single machine with a single Python interpreter; they reflect relative performance differences between the tested classes, but not absolute, hardware-agnostic, and consistent throughputs. The tracemalloc memory figures reflect only Python-allocated memory and not the total footprint of the process. Sixth, the controlled scaling benchmark only runs on prefixes of the 22,309-row test split, and while it shows a near-linear trend within this range, there is no guarantee of this trend holding for significantly larger graphs, such as IBM's data set, which was only benchmarked for a single 24-hour period. Lastly, several of the 33 knowledge base rules use

thresholds this project selected, rather than a direct quote from a regulatory source; the catalog explicitly notes this for each rule, while the regulatory guidance should be read as an endorsement of the patterns themselves, rather than any particular quantitative thresholds FinReason may apply.

9. Issues Faced During Development

Several issues arose during development that saw changes to the design or additions to the testing procedures. The first was a discovery about leakage; an early version of the pipeline had an implicit assumption that the chronological cut-off point contained label information. This was fixed by using exclusively event time and row count based balancing of the cut off, with an explicit assertion checked at notebook start that no anomaly/laundering column is present in a feature frame passed to any of the detectors or the statistical model fitting routines.

The second issue was one of distinguishing between a true improvement in detection power and an artefact of test design the naive versus contextual BFS/DFS comparison exists because an early version of the fan-out detector, tested only on positive scenarios, appeared to work well on all cases until the payroll and merchant hard-negative scenarios were introduced to the synthetic test set, at which point it became apparent that the shape only detector was flagging every single legitimate payroll run and merchant incoming payment as suspicious. It became necessary to include hard-negative examples in the test set in addition to the positive examples for any detector that had to distinguish between them.

The third issue was one of memory management: an early attempt to load the whole 5.08M row IBM AML dataset into the memory as a MultiDiGraph was unfortunately too large for the 8 GiB RAM of the test machine after accounting for node and edge attribute storage overhead. The fix consisted of restructuring the graph around a sliding-window paradigm, streaming the source file in 250,000 row chunks and discarding the windows graphs after benchmarking, which is reflected in the implementation notes of Section 4 and the associated memory consumption estimates of Section 7. A fourth issue consisted of an implicit column-suffix collision while joining the transaction-level and scenario-level test metric tables during evaluation; two columns with the same name originating from different frames would cause a dataframe with a name suffixed column to be output if the join wasn't performed with an explicit validation argument. This was fixed by adding a schema safety validation step for any such joins, along with an explicit one-to-one validation and an assertion that no suffixed columns exist in the joined dataframe. Finally, adding a stable activation key (rule, subject and the exact set of parent fact IDs) to the forward-chaining engine allowed for rule firings on the same evidence to be marked as idempotent; this fix is reflected in the test procedure of the idempotence check, which re-runs the completed engine and asserts that it inserts no facts.

10. Conclusion

FinReason represents a hybrid approach to explainable financial anomaly detection, combining both graph search limited DFS/BFS and domain-specific rule engines evaluated against two statistical baselines in a single frozen, held-out evaluation procedure. On the synthetic minority oversampled test set, the approach achieves a precision/recall balance (F1 score) of 0.8399 (98.54%) across all scenarios through graph search alone, with the rule engine adding a narrower but substantially more precise escalation layer with full evidential provenance, and the combination of both detectors resulting in maximal but substantially higher false-positive rates for the given scenario set a precision/recall trade-off characteristic of most anomaly detection systems, but particularly evident due to the fixed, known-positive test set. Sixty automated tests, eleven integration checks and five determinism checks were passed in development, and all reported error counts matched the relevant confusion matrices for all detectors at all stages. This gives a high degree of confidence that the results described in this paper reflect the implemented solution accurately. However, the limitation of this approach is one of scale: the results described are based on synthetic and single-machine processing benchmarks, and the paper has attempted to qualify results that may be extrapolated beyond the described scenario set rather than suggesting an immediate generalization to all financial anomaly detection scenarios.

11. Project Proposal, Code and Demo

To run the code local environment, execute following commands in order from the root folder.

- Step 1 - Create and activate a virtual environment

  python3 -m venv .venv

  source .venv/bin/activate

- Step 2 - Upgrade pip and install dependencies

  python -m pip install --upgrade pip

  python -m pip install -r requirements.txt

- Step 3 - Run the project

  python scripts/run_project.py

Source code drive link -

https://drive.google.com/file/d/1dcPvlZqseBHKzZIkcr2DLXCCCn44aliZ/view?usp=sharing

Demo video drive link –

https://drive.google.com/file/d/1K0d3vsabtEZhxHelG9gIYFG1hu_XYVsH/view?usp=sharing

Project Proposal -

https://docs.google.com/document/d/179OXxlb3MV8ukzlLehp0nZigvq8iAvUL/edit?usp=sharing&ouid=104945321378718381713&rtpof=true&sd=true

12. References

Akoglu, L., Tong, H. & Koutra, D., 2015. Graph based anomaly detection and description: a survey. *Data Mining and Knowledge Discovery.*

Alarab, I., Prakoonwit, S. & Nacer, M. I., 2022. Graph-based LSTM for anti-money laundering: experimenting temporal graph convolutional network with bitcoin data. *Neural Processing Letters,* Volume 54.

Altman, E. et al., n.d. *Realistic synthetic financial transactions for anti-money laundering models.* s.l., s.n.

Bellomarini, L., Laurenza, E. & Sallinger, E., 2020. *Rule-based anti-money laundering in Financial Intelligence Units: Experience and vision.* s.l., s.n.

Cardoso, M., Saleiro, P. & Bizarro, P., n.d. *LaundroGraph: self-supervised graph representation learning for anti-money laundering.* s.l., s.n.

Cheng, D. et al., 2024. Anti-money laundering by group-aware deep graph learning. *IEEE Transactions on Knowledge and Data Engineering.*

Chen, Z. et al., 2023. Parallel path detection for fraudulent accounts in banks based on graph analysis. *PeerJ Computer Science.*

Deprez, B. et al., 2025. Network analytics for anti-money laundering: a systematic literature review and experimental evaluation. *INFORMS Journal on Data Science.*

Dilini, N., Sun, N., Miao, Y. & Moustafa, N., 2026. A comprehensive review on graph-based anomaly detection: Approaches for intrusion detection. *Applied Sciences,* Volume 16.

Dumitrescu, B., Bălțoiu, A. & Budulan, Ș., 2022. Anomaly detection in graphs of bank transactions for anti money laundering applications. *IEEE Access,* Volume 10.

Hilal, W., Gadsden, S. A. & Yawney, J., 2022. Financial fraud: A review of anomaly detection techniques and recent advances. *Expert Systems with Applications,* Volume 193.

Lopez-Rojas, E., Elmir, A. & Axelsson, S., 2016. *PaySim: A financial mobile money simulator for fraud detection.* s.l., s.n.

Motie, S. & Raahemi, B., 2024. Financial fraud detection using graph neural networks: a systematic review. *Expert Systems with Applications,* Volume 240.

Pourhabibi, T., Ong, K.-L., Kam, B. H. & Boo, Y. L., 2020. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. *Decision Support Systems.*

Priyadarshi, S., Kumar, J. & Arunachalam, N., 2025. *Financial fraud detection using graph neural networks.* s.l., s.n.